

Код-ревью дипломной работы

Репозиторий: https://github.com/ilyared89/11.HW_JS_Playwright_API_Diplom

Студент: Илья Поединков

Дата проверки: 2026-06-07

Проверяющий: mnmlsniper, Умница AI Code Reviewer

Работа отклонена — критичные проблемы требуют исправления

Проект содержит хорошую техническую основу: реальный json-server через `globalSetup`, семантические локаторы, чистую CI/CD конфигурацию. Вместе с тем обнаружены 5 критичных проблем, которые блокируют принятие. Главная из них — отсутствие паттерна Facade, который является ключевым требованием задания. Дополнительно: Builder-паттерн реализован, но не применяется в тестах; хардкод учётных данных; позитивные API-тесты не проверяют все поля ответа.

Детальные рекомендации и минимальные требования для принятия — в разделе «Резюме и требования для принятия» в конце отчёта.

Статистика замечаний

Всего замечаний: 10

По приоритетам:

-  Критичные: 5
-  Высокий: 1
-  Средний: 2
-  Низкий: 2

По типам проблем:

- Архитектура (Facade): 1
- API тесты (тест-дизайн, валидация): 4
- Безопасность: 1
- Builders (паттерн): 1

- UI тесты (утверждения): 1
 - Общие (стиль, документация): 2
-

Общая сводка

Структура проекта

```
11.HW_JS_Playwright_API_Diplom/
├─ src/
│  └─ helpers/
│     └─ builders/      # UserBuilder, PostBuilder, CommentBuilder +
index.js
│     └─ fixtures/      # ui.fixture.js, api.fixture.js
│     └─ pages/          # BasePage, LoginPage, RegisterPage, HomePage,
ProductPage, CartPage + index.js
│     └─ services/       # ApiService, PostsService, CommentsService +
index.js
├─ tests/
│  └─ api/               # 5 файлов (posts, get, update, delete, comments)
│  └─ ui/                # 5 файлов (login, register, search, newsletter,
cart)
│     └─ setup/          # global.setup.js
├─ allure/               # categories.json
├─ .github/workflows/    # ci.yml
└─ playwright.config.js
```

Назначение: Дипломный проект. UI тесты — demowebshop.tricentis.com (авторизация, регистрация, поиск, newsletter, корзина). API тесты — локальный json-server (CRUD для posts и comments).

Технологии: Playwright, JavaScript (ESM), Faker.js, Allure, dotenv, json-server.

Паттерны: POM с наследованием (BasePage), Builder с fluent API, Fixtures.

Количество тестов: 10 UI + 7 API = 17 тестов.

CI/CD: GitHub Actions → Allure Report на GitHub Pages + Telegram-уведомления через GitHub Secrets.

Что сделано хорошо

Builders — структура — Все три Builder-класса корректно реализованы как классы: конструктор инициализирует только пустой объект, данные генерируются в методах (`addEmail()`, `addPassword()` и т.д.). Fluent API с `withXxx()` методами для переопределения значений реализован правильно. `UserBuilder` корректно синхронизирует `password` и `confirmPassword`. Созданы `index.js` в `builders/`, `pages/`, `services/` — единая точка импорта для каждого слоя. (Применение паттерна в тестах — см. замечание #6.)

Реальный json-server через globalSetup — `global.setup.js` динамически выделяет свободный порт, стартует сервер, сохраняет URL в `process.env.API_BASE_URL`, а `globalTeardown` корректно останавливает его. Каждый запуск начинается с чистых seed-данных. Решение профессиональное.

expect() только в теле теста — `Login`, `Register`, `Home`, `Cart`, `Product Pages` содержат только геттеры локаторов. `PostsService` и `CommentsService` содержат только HTTP-методы. `Assert`-секция присутствует исключительно в теле теста.

Изоляция API тестов — `DELETE`-тест создаёт пост, удаляет по полученному `id` и верифицирует 404 при повторном `GET`. `PUT` и `PATCH` тесты также создают собственные данные перед обновлением. Зависимости от внешнего состояния устранены.

Allure и CI/CD — Глубокая интеграция с Allure (`epic`, `feature`, `story`, `severity`, скриншоты). GitHub Actions с передачей секретов через GitHub Secrets, деплой отчёта в GitHub Pages, Telegram-уведомления. Кэширование npm-зависимостей.

Замечания

Критичные проблемы (ОТКЛОНЯЮТ РАБОТУ)

1. Паттерн Facade не реализован — ни для UI, ни для API

Приоритет:  Критичный

Описание: Facade — один из ключевых паттернов данного задания. Его назначение: единая точка входа для работы с UI или API, которая скрывает набор отдельных Page Objects или сервисов за одним объектом, предоставляемым тесту через фикстуру.

UI. В `ui.fixture.js` реализованы отдельные фикстуры для каждой страницы: `loginPage`, `registerPage`, `homePage`, `productPage`, `cartPage`. Это набор изолированных объектов, а не Facade. Тесты работают с пятью разными фикстурами

вместо одной точки входа. Отсутствует класс, агрегирующий все страницы — нет ни файла, ни класса, который играл бы роль `App -Facade`.

API. `api.fixture.js` предоставляет фикстуру `api` (базовый HTTP-слой `ApiService`), а также отдельные `postsApi` и `commentsApi`. Тесты обращаются к `postsApi.create()`, `commentsApi.getByPost()` напрямую через разные фикстуры. Отдельного класса-агрегатора (`Api`), который объединял бы `.posts` и `.comments` под одной точкой входа, нет.

В результате тесты не могут использовать единый объект для доступа ко всему UI или всему API — это противоречит требованию Facade как паттерна и критерию задания.

Исправление: Реализуйте Facade-класс для UI (например, `App`) и Facade-класс для API (например, `Api`), каждый из которых агрегирует соответствующие Page Objects или сервисы в виде свойств. Предоставляйте эти классы через фикстуры (`app`, `api`) — тест должен получать один объект и обращаться к нужной странице или сервису через него.

Файл	Контекст
<code>src/helpers/fixtures/ui.fixture.js</code>	Пять отдельных фикстур страниц вместо одного <code>app -Facade</code>
<code>src/helpers/fixtures/api.fixture.js</code>	<code>postsApi</code> и <code>commentsApi</code> как отдельные фикстуры вместо единого <code>api -Facade</code>
<code>tests/ui/*.test.js</code>	Тесты деструктурируют отдельные страницы (<code>{ loginPage }</code> , <code>{ homePage, productPage, cartPage }</code>)
<code>tests/api/*.test.js</code>	Тесты используют <code>{ postsApi }</code> / <code>{ commentsApi }</code> напрямую

2. Хардкод учётных данных в `login.test.js`

Приоритет: 🔴 Критичный

Описание: В позитивном тесте авторизации жёстко прописаны реальные учётные данные:

```
// tests/ui/login.test.js, строка 14
await loginPage.login("testuser@example.com", "TestPassword123");
```

Хардкод паролей в коде — серьёзная проблема безопасности вне зависимости от среды. Если аккаунт изменится, тест сломается без понимания причины. Email и пароль фигурируют вместе в открытом репозитории, что фактически публикует учётные данные.

Исправление: Перенесите учётные данные в переменные окружения и добавьте их в `.env.example` с placeholder-значениями:

```
# .env.example
DEMO_EMAIL=your_test_email@example.com
DEMO_PASSWORD=your_test_password
```

```
// tests/ui/login.test.js
await loginPage.login(process.env.DEMO_EMAIL, process.env.DEMO_PASSWORD);
```

```
# .github/workflows/ci.yml
env:
  DEMO_EMAIL: ${ secrets.DEMO_EMAIL }
  DEMO_PASSWORD: ${ secrets.DEMO_PASSWORD }
```

Файл	Строка	Контекст
tests/ui/login.test.js	14	loginPage.login("testuser@example.com", "TestPassword123")

3. Позитивный тест создания поста не проверяет все поля ответа

Приоритет:  Критичный

Описание: Тест 'Creates a new post' отправляет объект с тремя полями (`title` , `body` , `userId`), но проверяет только `title` и `id` . Поля `body` и `userId` не верифицируются. В позитивных тестах необходимо проверять все поля ответа — это обязательное требование тест-дизайна.

```
// Текущее состояние:
const post = newPost(); // {title, body, userId}
// ...
expect(body).toHaveProperty("title", post.title); // 
expect(body).toHaveProperty("id"); // 
//  Не проверяются: body.body, body.userId
```

Файл	Строка	Контекст
tests/api/posts.test.js	20–21	Проверяются только title и id; body и userId пропущены

4. Позитивный тест обновления (PUT) не проверяет все поля ответа

Приоритет:  Критичный

Описание: Тест 'Updates post with PUT' отправляет полный объект с тремя полями (title, body, userId), но в ответе проверяется только title. Поля body и userId не верифицируются. PUT — это полная замена ресурса, поэтому необходимо убедиться, что все поля были обновлены корректно.

```
// Текущее состояние:
const update = newPost(); // {title, body, userId}
// ...
expect(body.title).toBe(update.title); // 
//  Не проверяются: body.body, body.userId
```

Файл	Строка	Контекст
tests/api/posts-update.test.js	27	expect(body.title).toBe(update.title) — body и userId не проверяются

5. Позитивный тест создания комментария не проверяет все поля ответа

Приоритет:  Критичный

Описание: Тест 'Creates comment for post' отправляет объект с тремя полями (name, email, body) и привязывает комментарий к посту по postId=1 через URL. json-server при создании вложенного ресурса через /posts/1/comments автоматически включает postId в тело ответа. Поле postId не верифицируется в assertions.

```
// Текущее состояние:
const comment = newComment(); // {name, email, body}
const res = await commentsApi.create(1, comment); // postId=1 в URL
```

```
// ...
expect(body.name).toBe(comment.name);    // ✓
expect(body.email).toBe(comment.email);  // ✓
expect(body.body).toBe(comment.body);    // ✓
expect(body.id).toBeDefined();           // ✓
// ✗ Не проверяется: body.postId
```

Дополнительно: значение `postId ( 1 )` захардкожено в вызове `commentsApi.create(1, comment)`. Тест привязан к конкретному посту из `seed`-данных — при изменении данных тест может некорректно работать или проверять не тот ресурс.

Файл	Строка	Контекст
tests/api/comments.test.js	21	<code>commentsApi.create(1, comment)</code> — <code>postId</code> захардкожен
tests/api/comments.test.js	25–28	<code>postId</code> из ответа не проверяется

● Высокий приоритет (ОТКЛОНЯЮТ РАБОТУ)

6. Builder pattern не применяется в тестах

Приоритет: ● Высокий

Описание: Builder-классы (`PostBuilder`, `CommentBuilder`, `UserBuilder`) реализованы структурно корректно, однако в тестах они не используются напрямую. Вместо пошаговой сборки объекта через цепочку методов тесты везде используют фабричные функции-обёртки:

```
const post = newPost();           // new PostBuilder().build() – автосборка
const comment = newComment();     // new CommentBuilder().build() – автосборка
const user = newUser();           // new UserBuilder().build() – автосборка
```

Смысл паттерна Builder — возможность гибко конструировать объекты в теле теста через цепочку вызовов. `newPost()` — это фабрика, а не Builder. Тест, использующий только `newPost()`, не демонстрирует паттерн. Методы `withXxx()` (для задания конкретных значений в тест-сценариях) нигде в тестах не применяются — хотя именно они дают практическую ценность паттерна.

Исправление: Замените `newPost()`, `newComment()`, `newUser()` в тестах на явные цепочки Builder-методов. `withXxx()` используйте там, где тест требует конкретных значений.

Файл	Контекст
<code>tests/api/posts.test.js</code>	<code>const post = newPost()</code> — Builder не применяется
<code>tests/api/posts-update.test.js</code>	<code>newPost()</code> для создания и обновления — Builder не применяется
<code>tests/api/comments.test.js</code>	<code>const comment = newComment()</code> — Builder не применяется

🟡 Средний приоритет (не блокируют принятие)

7. GET single post — неполные assertions

Описание: Тест `'Gets a single post'` проверяет только `id` и `title.isDefined()`. Поля `body` и `userId` не проверяются — даже как `toBeDefined()`. Для seed-данных с известными значениями можно проверить точные значения или хотя бы наличие всех полей.

Рекомендация:

```
expect(body.id).toBe(1);
expect(body.title).toBeDefined();
expect(body.body).toBeDefined();
expect(body.userId).toBeDefined();
```

Файл	Строка	Контекст
<code>tests/api/posts-get.test.js</code>	25–26	Проверяются только <code>id</code> и <code>title</code> ; <code>body</code> , <code>userId</code> — нет

8. Позитивный тест логина не проверяет идентичность пользователя

Описание: После успешного логина тест проверяет только видимость элемента

`.account :`

```
await expect(loginPage.accountHeaderLocator).toBeVisible();
```

Это подтверждает, что *какой-то* пользователь вошёл, но не верифицирует конкретного пользователя, чьи данные были использованы.

Рекомендация:

```
await expect(loginPage.accountHeaderLocator).toBeVisible();
await
expect(loginPage.accountHeaderLocator).toContainText(process.env.DEMO_EMAIL.
split('@')[0]);
```

Файл	Строка	Контекст
tests/ui/login.test.js	15	toBeVisible() — не верифицирует конкретного пользователя

Низкий приоритет (не блокируют принятие)

9. Неиспользуемый `import { expect } в base.page.js`

Описание: `base.page.js` импортирует `expect` из `@playwright/test`, но нигде в классе его не использует.

Рекомендация: Удалить строку 1 из `base.page.js`.

Файл	Строка	Контекст
src/pages/base.page.js	1	<code>import { expect } from "@playwright/test"</code> — не используется

10. README упоминает устаревший `my-json-server.typicode.com`

Описание: В таблице технологического стека README указан `my-json-server.typicode.com`, хотя фактически используется локальный `json-server`.

Рекомендация: Обновить строку таблицы: `json-server (local, via globalSetup)`.

Файл	Строка	Контекст
README.md	~24	Таблица tech stack — устаревшее значение

Проверка безопасности

Критическая проверка

```
grep -r -i "password\|token\|api_key\|secret" --include="*.js" .
```

- ❌ Хардкод пароля в `login.test.js` — `"TestPassword123"` в строке 14
- ❌ Хардкод email в `login.test.js` — `"testuser@example.com"` в строке 14
- ✅ Хардкод в Builders — не обнаружено (только faker-генерация)
- ✅ API ключи в JS-файлах — не обнаружено
- ✅ Токены в коде — не обнаружено

Проверка переменных окружения

- ✅ `playwright.config.js` использует `process.env.BASE_URL` — правильно
- ✅ `api.fixture.js` использует `process.env.API_BASE_URL` || `"http://localhost:3000"` — правильно
- ✅ `.env` в `.gitignore` — правильно
- ❌ Учётные данные для `login.test.js` должны быть в `process.env.DEMO_EMAIL` / `DEMO_PASSWORD`

Проверка логирования

- ✅ `console.log` в `global.setup.js` — только технические сообщения о запуске/остановке сервера, не содержат чувствительных данных
- ✅ Логирование токенов и паролей — не обнаружено

Проверка конфигурации

- ✅ `ci.yml` использует `${{ secrets.TELEGRAM_BOT_TOKEN }}`, `${{ secrets.TELEGRAM_CHAT_ID }}` — правильно

- ✅ GitHub Secrets для sensitive данных — правильно настроено
- ✅ Чувствительные данные (токены, ключи) — не обнаружены в tracking-файлах

Сводная оценка

Критерий	Оценка	Комментарий
UI тесты — Селекторы	✅ Хорошо	<code>getByLabel</code> , <code>getByRole</code> , <code>getByText</code> — семантические локаторы
UI тесты — Ожидания	✅ Хорошо	<code>waitForTimeout</code> отсутствует
UI тесты — Утверждения	⚠️ Частично	Логин проверяет только видимость, не идентичность пользователя
UI тесты — POM	⚠️ Требуется исправления	Facade-класс <code>App</code> не реализован (🔴)
UI тесты — Фикстуры	⚠️ Требуется исправления	Индивидуальные PO вместо единого <code>app</code> - Facade (🔴)
UI тесты — Структура	✅ Хорошо	5 файлов по функциональности
API тесты — Контроллеры	⚠️ Требуется исправления	Facade-класс <code>Api</code> не реализован, <code>postsApi</code> / <code>commentsApi</code> — отдельные фикстуры (🔴)
API тесты — Валидация	⚠️ Требуется исправления	<code>CREATE posts</code> , <code>PUT</code> , <code>CREATE comments</code> не проверяют все поля ответа (🔴)
Тест-дизайн	⚠️ Требуется исправления	Позитивные тесты API неполные (🔴)
Builders	⚠️ Требуется исправления	Структура корректна, но в тестах используется как фабрика — паттерн не демонстрируется (🟡)
Безопасность	⚠️ Требуется исправления	Хардкод <code>TestPassword123</code> в <code>login.test.js</code> (🔴)
CI/CD	✅ Хорошо	GitHub Actions, <code>json-server</code> через <code>globalSetup</code> , Allure Pages, Telegram
Конфигурация	✅ Хорошо	<code>actionTimeout</code> , <code>navigationTimeout</code> , <code>retries</code> , <code>env vars</code>
Чистота репозитория	✅ Хорошо	<code>.env</code> , <code>db.json</code> , <code>secrets</code> не попадают в репозиторий

Наблюдение по процессу подготовки работы

В ходе проверки зафиксированы признаки использования инструментов генерации кода при подготовке работы:

- **Несоответствие уровня сложности.** `global.setup.js` содержит нетривиальный код: динамический поиск свободного порта через `node:net`, программный запуск `json-server` с `lifecycle`-управлением, генерация `Allure environment.properties`. Это значительно выходит за рамки тем курса — и при этом в той же работе отсутствует базовый `Facade`, который разбирался на занятиях.
- **Самоссылочные комментарии в коде.** `user.builder.js` строка 33 содержит `//`  Исправление бага #8 — внутренняя ссылка на номер замечания из чужого ревью. Такой комментарий не возникает при самостоятельном написании кода с нуля; он указывает, что код генерировался итерационно по чужим правкам.
- **Пропуск архитектурного паттерна.** `Facade` — тема курса. Его отсутствие при наличии технически сложного кода (динамический порт, `json-server lifecycle`, `Allure CI pipeline`) говорит о том, что код не писался из понимания архитектуры: сложные части заимствованы, базовые — пропущены.

Рекомендуем провести устную защиту или задать уточняющие вопросы по архитектурным решениям, прежде чем принять работу.

Резюме и требования для принятия

Работа **отклонена** — обнаружены 5 критичных (🔴) и 1 высокая (🟡) проблема. Для принятия все они должны быть устранены.

Критические (🔴) — обязательны к исправлению

1. **Паттерн `Facade` не реализован** — создайте `Facade`-класс для UI (агрегирует все `Page Objects`) и `Facade`-класс для API (агрегирует все сервисы); предоставляйте их через фикстуры `app` и `api` соответственно
2. **Хардкод учётных данных в `login.test.js`** — перенесите `testuser@example.com` и `TestPassword123` в переменные окружения (`process.env.DEMO_EMAIL`, `process.env.DEMO_PASSWORD`); добавьте их в `.env.example` как `placeholder` и в `GitHub Secrets`
3. **Позитивный тест `CREATE` поста** (`tests/api/posts.test.js`) — добавьте проверки `body.body` и `body.userId` в дополнение к существующим `title` и `id`

4. **Позитивный тест PUT обновления** (`tests/api/posts-update.test.js`) — добавьте проверки `body.body` и `body.userId` в дополнение к существующей `body.title`
5. **Позитивный тест CREATE комментария** (`tests/api/comments.test.js`) — добавьте проверку `body.postId` (ожидаемое значение: 1)

Высокие (🟡) — обязательны к исправлению

6. **Builder не используется как паттерн в тестах** — замените вызовы `newPost()`, `newComment()`, `newUser()` на явные цепочки Builder-методов; используйте `withXxx()` для задания конкретных значений в тест-сценариях

Рекомендации для улучшения (не блокируют принятие)

- Добавить `toBeDefined()` для `body` и `userId` в тест 'Gets a single post'
- Добавить проверку содержимого account header в позитивный тест логина
- Удалить неиспользуемый `import { expect } из base.page.js`
- Обновить README: заменить `my-json-server.typicode.com` на `json-server (local, via globalSetup)`

Репозиторий: https://github.com/ilyared89/11.HW_JS_Playwright_API_Diplom